HEXAGON
Digital Services

92 Days

FULL STACK
Web Development

Course Curriculum

MERN

Enroll Now

www.hexagondigitalservices.com

Prerequisites ——

01 HTML

2 - Day's 4hrs

KEY TOPICS:

1. **HTML Tags & Elements**
   - Headings (<h1>-<h6>), Paragraphs (<p>) , Lists (<li>, <ul>,<ol>), Links (<a>)
   - Semantic tags: <section>, <article>, <nav>, <footer>
   - Global attributes: id, class
2. **Forms & Input Elements**
   - <input>, <select>, <textarea>, <button>
   - Form validation attributes (required, type, minlength etc.)
3. **Media Elements**
   - <img>, <audio>, <video>
   - attributes: controls, autoplay, loop, muted
4. **Table and Layout Tags**
   - <table>, <thead>, <tbody>, <tr>, <td>

---

02 CSS

4 - Day's 8hrs

KEY TOPICS:

1. **Selectors**

- Basic Selectors: element, .class, #id
- Attribute Selectors: [type="text"], [href^="https"]
- Pseudo-classes: :hover, :focus, :nth-child()
- Pseudo-elements: ::before, ::after

2. **Styling**

- Typography: font-family, font-size, font-weight, line-height
- Colors: color, background-color, rbga(), gradients
- Borders and Shadows
- Transitions & Animations: transition, @keyframe, translation

3. **Positioning & Layout**

- position: static, relative, absolute, fixed, sticky
- display: block, inline, none, flex, grid
- Float, clear
- z-index

4. **Flexbox & Grid**

- parent: display: flex, justify-content, align-items, flex-direction
- child: flex-grow, flex-shrink, align-self

5. **Responsive Design**

- **Media Queries: @media (max-width: 768px) {...}**
- **Units: %, em, vw, vh**
- **Responsive Images: max-width, height: auto**

Mini Project 1

---

03 JavaScript

10 - Day's 20hrs

KEY TOPICS:

1. **Fundamentals**

- Variables: let, const, var
- Data Types:
  i. Primitive: String, Number, Boolean, Null, Undefined
  ii. Complex: Object, Array, Function
- Operators: +, -, %, *, !, ==, !=, ===, typeof, instanceof

2. **Functions & Scope**

- Function declaration, parameters, return
- Arrow functions, Higher-order functions: Callbacks, Functions returning functions
- Scope: Global, Local, Block, Hoisting
- Closures, Lexical scope
- Array methods: .map(), .filter(), .reduce()

3. **DOM Manipulation**

- Selecting Elements: getElementById(), querySelector()
- Modifying DOM: innerHTML, createElement(), appendChild(), removeChild(),
- classList
- Events: addEventListener(), click, submit, Event object

4. **Objects & Arrays**

- Object Literals: Properties, Methods, this, Destructuring, Spread operator
- Arrays:
  i. Methods: .push(), .pop(), .slice(), .splice()
  ii. Iteration: .forEach(), .find(), .some(), .every()
- JSON: JSON.parse(), JSON.stringify()

5. **Asynchronous JavaScript**

- Timers: setTimeout(), setInterval()
- Promises: .then(), .catch(), .finally()
- async/await syntax
- Fetch API: fetch(), GET/POST requests, Handling JSON

6. **Error Handling**

- try/catch/finally
- Error types: ReferenceError, TypeError, etc.

7. **Browser Storage**

- localStorage, sessionStorage

Mini Project 2

03 Tailwind CSS

4 - Day's 8hrs

KEY TOPICS:

1. **Tailwind Setup (CDN & CLI)**
2. **Utility-First CSS approach**
3. **Spacing, Typography, Colors, Shadows**
4. **Flexbox & Grid Utilities**
5. **Responsiveness in Tailwind (breakpoints, mobile-first design)**
6. **Common UI Components (Navbar, Cards, Buttons, Modals)**
7. **Customizing Tailwind (config file, themes, plugins)**

Mini Project 3

MERN Core Technologies

01 React.js (Frontend Library) 12 - Day's 24hrs

KEY TOPICS:

1. **JSX (JavaScript XML)**
   - Combines HTML with JavaScript syntax.
   - Must return a single parent element.

2. **Components :**

   Components are like functions that return HTML elements.
   - Functional component and class component
   - Props: passing data to components

3. **Hooks**
   - useState() – Manage local component state
   - useEffect() – Side effects and lifecycle methods
   - useRef() – Access DOM or persist values
   - useContext() – Global state management
   - useReducer() – Alternative to useState for complex logic
   - useMemo() / useCallback() – Performance optimizations
   - Custom Hooks – Encapsulate logic in reusable functions

4. **Routing**

5. **Forms and Events**

6. **State Management**

- Context API
- API Integration (Fetch & Axios)
- Redux

7. **Performance Optimization**

- useMemo useCallback
- virtualization

8. **Advanced:**

- Custom Hooks, React Fragments, Portals, Code Splitting, Lazy Loading

Mini Project 4

---

02 Backend with Node.js & Express.js 8 - Day's 16hrs

KEY TOPICS:

Node.js (JavaScript Runtime)

1. **Introduction to Node.js**
   - Event-driven, non-blocking I/O model
   - npm (Node Package Manager) for managing dependencies.
   - CommonJS modules (require & module.exports).
2. **Creating a Basic Server with Node.js**
3. **Environment variables**
   - Using dotenv for configuration.

Express.js (Backend Framework)

1. **Routing**
   - Handling HTTP methods (GET, POST, PUT, DELETE)
   - Route parameters (/users/:id)
   - Query parameters (/search?q=express)
2. **Middleware**
   - Body parsing middleware (express.json(), express.urlencoded())
   - Static file middleware (express.static())
   - Session middleware (express-session)
   - Third-party middleware
     i. CROS - Enables cros-origin requests
     ii. Multer - Handles file uploads
     iii. Express-session - Manages user sessions
     iv. Express-rate-limit - Prevents rate-based abuse

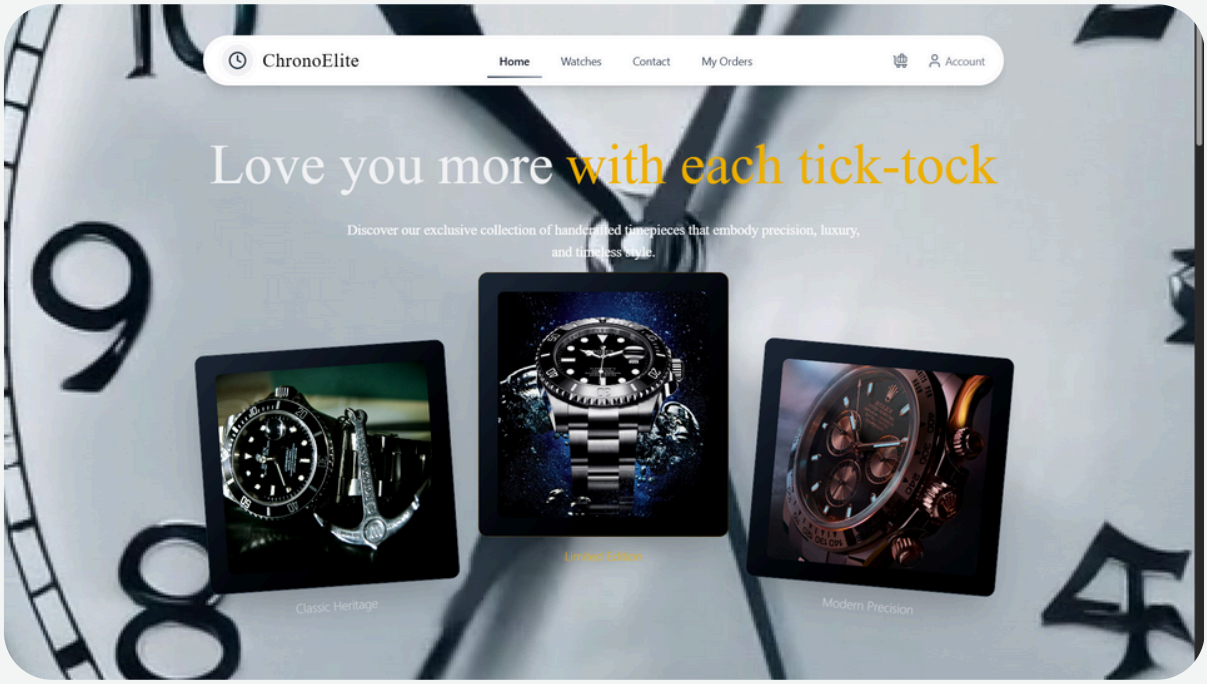
3. **Handling Request and Response**

- Access request data: req.params, req.query, req.body
- Send responses: req.send(), req.json(), req.status()

4. **Error Handling**

- app.use((err,req,res,next) => console.error(err.stack));

Mini Project 5



02 Database – MongoDB & Mongoose

2 - Day's 4hrs

KEY TOPICS:

1. **Introduction to MongoDB**

- **A document-based NoSQL database.**
- **Stores data in BSON (Binary JSON) format.**
- **Highly scalable with built-in sharding and replication.**

2. **Core Concepts**

- **Document Structure**
  i. A document is a key-value pair (similar to JSON)
  ii.

```
{
 "_id": "123",
 "name": "John Doe",
 "age": 30,
 "address": { "city": "New York", "country": "USA" }
 }
```

- **Collections & Databases**
  i. Collection : Group of documents (similar to a table in SQL)
  ii. Database : Contains multiple collections

3. **CRUD Operations**

4. **Indexing & Performance**

- **Single Field Index: db.users.createIndex({ name: 1 })**
- **Compound Index: db.users.createIndex({ name: 1, age: -1 })**
- **Text Index (for search): db.articles.createIndex({ content: "text" })**

6. **Querying Data :**

- **Basic Queries: find(), findOne().**
- **Comparison Operators: $eq, $ne, $gt, $lt, $in.**
- **Logical Operators: $and, $or, $not.**
- **Array Operations: $push, $pull, $elemMatch.**

7. **Aggregation Framework**

- **Pipeline stages: $match, $group, $sort, $project.**

KEY TOPICS:

1. **RESTful APIs and principles:**

REST (Representational State Transfer) is an architectural style for designing APIs using standard HTTP methods.

- Stateless: Each request contains all the data needed (no memory of previous requests).
- Resource-based: Resources are accessed via unique URLs (e.g., /api/users/1).
- JSON is the most common data format for requests and responses.

2. **HTTP Methods:**

Method	Description	Example
GET	Read data	`GET /api/products`
POST	Create a new resource	`POST /api/products`
PUT	Update an existing resource	`PUT /api/products/5`
DELETE	Remove a resource	`DELETE /api/products/5`

3. **Request & Response Breakdown:**
4. **API Testing Tools:**

- Postman, Thunder Client

04 Git & GitHub

1 - Day's 2hrs

KEY TOPICS:

1. **Version Control System**
2. **Middleware:**

- git init - initializes a new Git repository
- git clone <url> - copies a remote repository to your local machine
- git add <file> - stages changes for commit
- git commit -m "message" - saves changes with a message
- git status - shows modified/staged files
- git log - displays commit history

3. **Branching & Merging**

- git branch <branch-name> - creates a new branch
- git checkout <branch-name> - switches to the branch
- git merge <branch-name> - combines changes from one branch into another

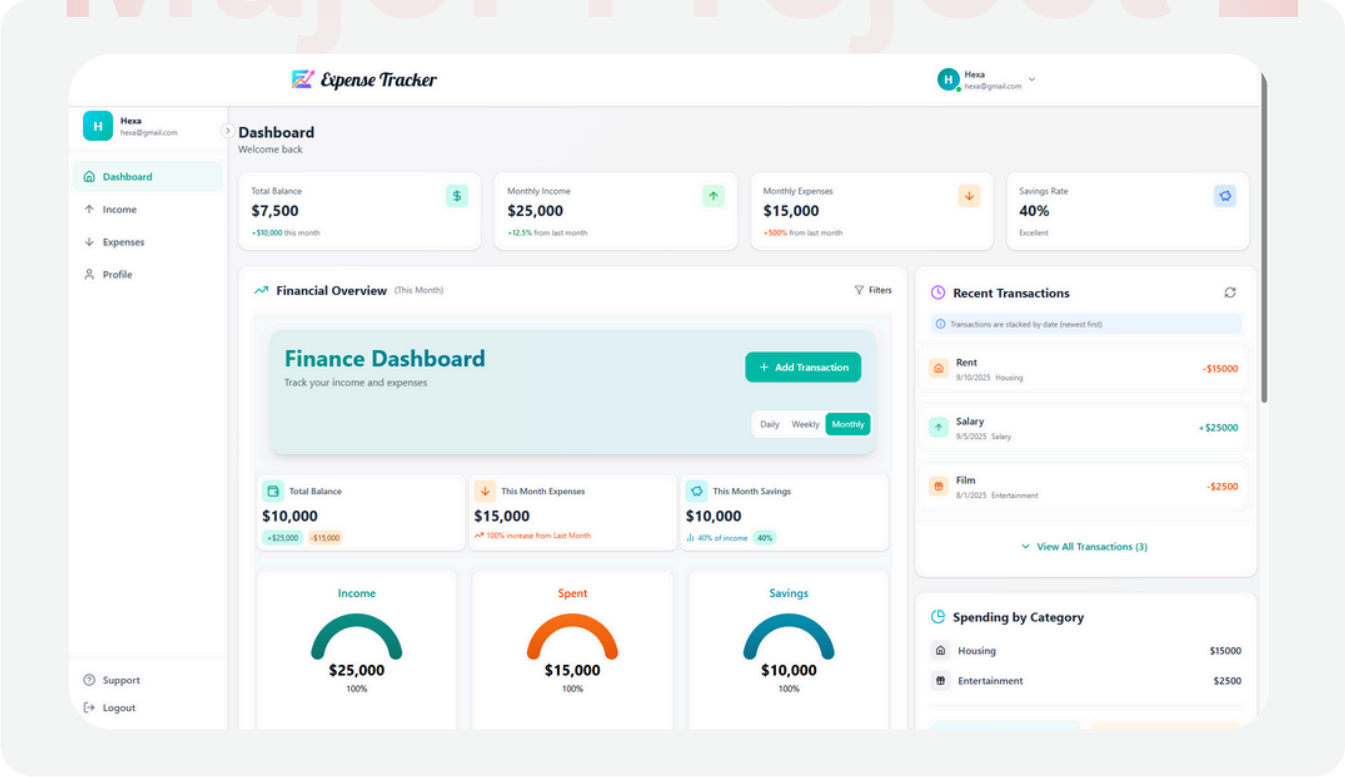
4. **Remote Repositories**

- git remote add origin <url> - links local repo to a remote GitHub repo
- git push -u origin main - uploads local commits to GitHub
- git pull - fetches remote changes and merges them
- git fetch- downloads remote changes without merging

Major Projects —— 45 Days

01 Expense Tracker 22 - Days

Major Project 1



About:

A full-stack expense tracker application where users can:

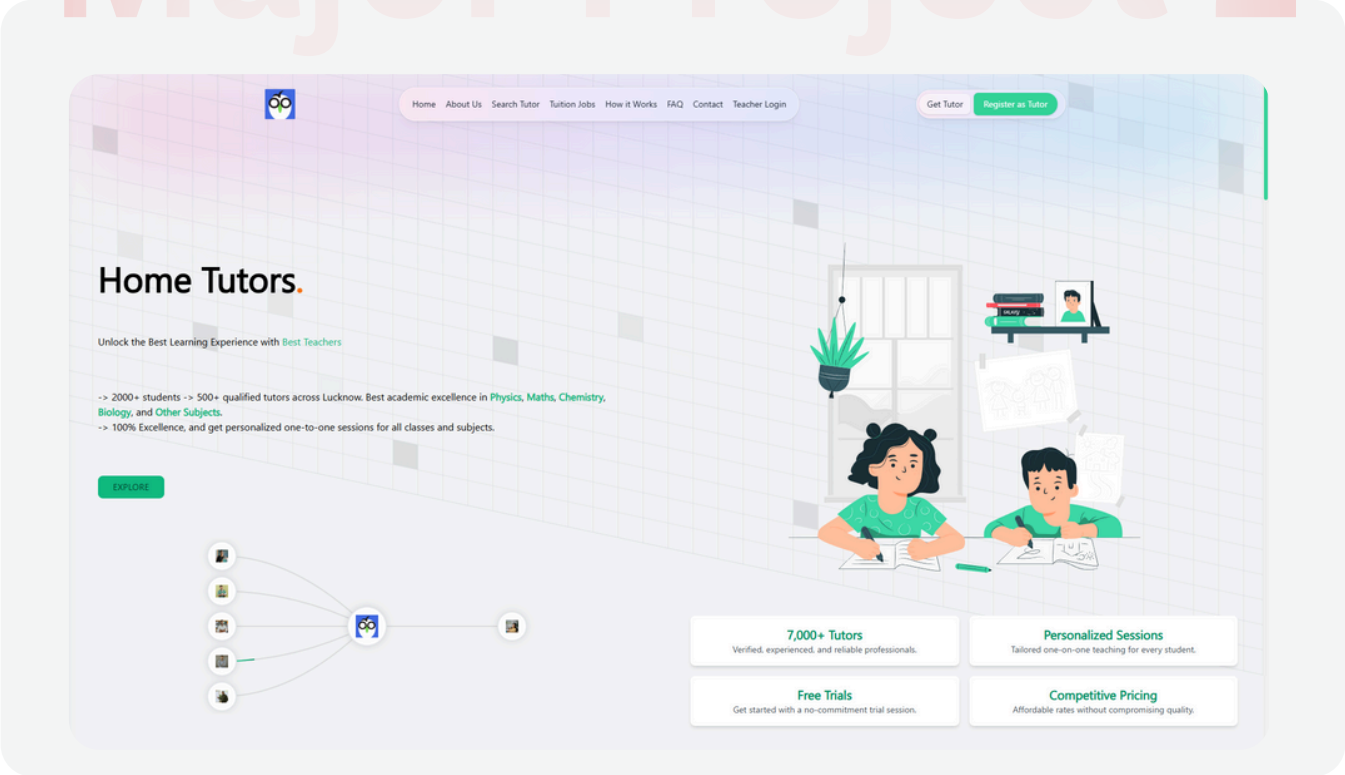
- Add, edit, and delete income and expense records
- Categorize transactions (e.g., Food, Travel, Bills, Salary)
- Track total income, expenses, and savings
- Visualize financial data with interactive graphs and charts
- View transaction history with filters for daily, weekly, monthly, and yearly insights
- Export their financial data to Excel for reporting
- Get smart recommendations with savings tags and month-to-month comparisons
- Secure authentication (Signup, Login, Logout)

KEY TOPICS:

1. **Expense Tracker App**

- Project Setup (React + Express + MongoDB)
- User Authentication (JWT-based Signup, Login, Logout)
- Transaction Model & API (CRUD for Income & Expenses)
- Expense Management UI (React + Tailwind)
- Data Filters (Daily, Weekly, Monthly, Yearly)
- Data Visualization (Interactive Graphs & Charts)
- Excel Export (Downloadable financial reports)
- Smart Recommendations (Savings tags & Month-to-month comparison)
- API Integration (Frontend–Backend connection)
- Testing & Debugging
- Deployment (Render)
- Final Touches + Documentation

Major Project 2



About:

A full-stack teacher booking platform where:

- **Teachers can register with a minimal fee via Stripe payment integration**
- **Create and manage their profile, visible to students/parents**
- **Fetch demo lecture video (YouTube/Google Drive link) to showcase teaching style**
- **Access job postings created by parents for their children**
- **Students/parents can send connect requests directly through a teacher's profile**
- **Unique IDs (UIDs) are generated for both teachers and students for easy profile search**
- **Students can create job forms and connect with teachers matching their**
- **requirements**
- **A Major Admin Panel (agency/owner) manages both tutor and student profiles, has access to hidden contacts, and facilitates direct connections between teachers and students**

KEY TOPICS:

1. **Teacher Booking for Home Tuition**

- Project Setup (React + Express + MongoDB)
- Tutor & Parent Authentication (JWT-based Signup, Login, Logout)
- Tutor Profiles (Subjects, Experience, Availability, Demo Lecture Video via YouTube/Drive link)
- Search & Filter (Find tutors by Subject, Location, or UID)
- Job Forms (Parents create & post tuition requirements)
- Connect Requests (Parents/students send requests to tutors)
- Booking & Access (Tutors access posted jobs; parents book tutors)
- Payment Gateway Integration (Stripe for tutor registration fee)
- Major Admin Panel (Manage tutor/student profiles, view hidden contacts, connect teachers with students)
- API Integration (Frontend–Backend connection)
- Testing & Debugging
- Final Deployment + Documentation/Demo

<Learn_Code_Deploy/>

HEXAGON

Digital Services

Enroll Now